DESIGN AND FUNCTIONAL VERIFICATION OF AN 8-BIT ALU WITH 16 OPERATIONS USING VHDL

SUBMITTED BY :

- **SNIGDHA BHARDWAJ**
- **SUYASH SHUKLA**

Abstract

The Arithmetic Logic Unit (ALU) is one of the most important components of a digital system, responsible for performing arithmetic and logical operations in processors and embedded systems. This project presents the design and functional verification of an 8-bit ALU with 16 operation choices using VHDL. The ALU performs various arithmetic, logical, and shift operations based on a 4-bit selection input. The design was developed using a modular approach and simulated to verify correct functionality for different input combinations. Test cases were created to validate each operation and ensure accurate outputs. The results demonstrate correct implementation of the ALU and provide practical exposure to digital design using VHDL and hardware description techniques used in modern VLSI systems.

Table of Contents

1. Introduction
2. Objectives
3. Background
    - Digital Systems and ALU
    - Role of ALU in Processors
4. Understanding FPGA (Field Programmable Gate Array)
5. Introduction to Xilinx Vivado
6. 8-Bit ALU Architecture
    - Block Diagram
    - ALU Operations (16 Operations)
7. Tools and Environment Used
8. Design Methodology
9. VHDL Implementation
    - ALU Design Code
    - Testbench Code
10. Simulation and Verification
11. Results and Output Waveforms
12. Future Scope / FPGA Implementation Possibility
13. Conclusion
14. References

Introduction

The Arithmetic Logic Unit (ALU) is one of the most important components of a digital system and forms the core of modern processors. It is responsible for performing arithmetic and logical operations on binary data. Operations such as addition, subtraction, logical AND, OR, and bit manipulation are all carried out by the ALU during program execution.

The purpose of this project is to design and simulate an 8-bit Arithmetic Logic Unit with 16 operations using VHDL. The design is developed using a hardware description language and implemented using the Xilinx Vivado design environment. By creating this ALU, the project demonstrates the fundamentals of digital circuit design, hardware modeling, and verification using simulation tools. This project also helps in understanding how digital systems can be described, tested, and prepared for hardware implementation.

Objectives:

The purpose of this project is:

• To design an 8-bit Arithmetic Logic Unit (ALU) using VHDL.
• To implement multiple arithmetic and logical operations using a selection input.
• To design a system capable of performing 16 different operations.
• To simulate and verify the functionality of the ALU using Vivado simulation tools.
• To gain practical knowledge of digital design concepts and FPGA-based development flow.

- ## Introduction to fpga

Understanding FPGA: Field-Programmable Gate Array

Field-Programmable Gate Arrays (FPGAs) are integrated circuits that can be configured by the user or designer after manufacturing. The term “field programmable” means that the device can be programmed in the field according to the required functionality. FPGAs consist of an array of programmable logic blocks and a network of reconfigurable interconnections that allow these blocks to be connected in different ways.

- ## Key Characteristics of FPGAs

1. Reconfigurability
FPGAs can be reprogrammed to perform different functions or to improve existing designs. This flexibility makes them useful for many different applications.

2. Parallel Processing
Unlike traditional processors that execute instructions sequentially, FPGAs can execute multiple operations simultaneously. This greatly increases efficiency for specific tasks.

3. Low Latency
FPGAs can process data with very low delay because the hardware is customized for the application, reducing unnecessary processing steps.

- ## FPGA Architecture

1. Configurable Logic Blocks (CLBs)
CLBs are the main logic components in an FPGA. They contain look-up tables (LUTs), flip-flops, and multiplexers which are used to implement different logic functions.

2. Interconnects
A programmable network of interconnects is used to connect the logic blocks. These connections allow signals to travel between different parts of the FPGA.

3. Input/Output Blocks (I/O Blocks)
I/O blocks allow the FPGA to communicate with external devices and other components in a system.

4. Memory Blocks
FPGAs include built-in memory that can be used for temporary data storage, buffering, and caching.

5. Clock Management
Clock management units such as phase-locked loops (PLLs) help control timing and synchronization of operations inside the FPGA.

- ## FPGA Development Process

1. Design Entry
The design process begins by describing the required functionality using hardware description languages such as VHDL or Verilog.

2. Simulation
The design is simulated to verify its behavior before

implementing it on hardware. Simulation helps detect errors early in the development process.

3. Synthesis
The HDL code is converted into a netlist that represents the hardware components and their connections.

4. Implementation
The synthesized design is mapped onto the FPGA resources such as logic blocks and interconnects. Placement and routing are performed in this stage.

5. Programming
A bitstream file is generated and used to configure the FPGA so that it performs the desired function.

6. Verification and Testing
After programming the FPGA, the design is tested to ensure that it operates correctly and meets performance requirements.

- **<u>Applications of FPGAs</u>**

1. **Telecommunications** – Used in routers, switches, and communication systems for high-speed data processing.
2. **Consumer Electronics** – Used in cameras, smartphones, and gaming systems for signal and image processing.
3. **Automotive Systems** – Used in driver assistance systems, infotainment units, and vehicle control modules.

Advantages of FPGAs

- Flexibility – Can be reprogrammed for different tasks.
- High Performance – Parallel processing allows faster computation.

Disadvantages of FPGAs

- Higher initial cost compared to ASICs for mass production.
- Power consumption can be higher than fixed hardware solutions.

The ALU designed in this project follows the same design methodology used in FPGA-based digital systems. By writing the logic in VHDL and verifying it through simulation, the design can be further synthesized and implemented on FPGA hardware.

- **<u>Introduction to VHDL Programming</u>**

VHDL (VHSIC Hardware Description Language), where VHSIC stands for *Very High Speed Integrated Circuits*, is widely used as a design-entry language for Field Programmable Gate Arrays (FPGAs) and Application-Specific Integrated Circuits (ASICs) in digital system design. It is commonly used in electronic design automation tools to describe and model digital circuits.

VHDL was originally developed for the United States Department of Defense to document the behavior of complex digital systems used in military equipment. Instead of relying on large and complicated manuals, VHDL provided a structured way to describe the functionality of hardware systems.

One major advantage of VHDL is that designs can be simulated before they are physically implemented in hardware. Logic simulators were developed to read VHDL files and verify the correctness of the design. Later, synthesis tools were introduced that could translate VHDL code into actual hardware implementations consisting of logic gates and interconnections.

Modern synthesis tools can automatically identify components such as memory blocks, counters, and arithmetic units from the VHDL description. These tools can optimize the design based on different requirements such as minimizing area, reducing power consumption, or achieving higher operating speed.

Once the design is created, it can be verified using simulation tools that display the waveform outputs of the circuit. A testbench is used to provide input signals and observe how the system behaves under different conditions. For example, a clock signal may be generated using loops or iterative statements in the testbench.

A key advantage of VHDL is that it allows designers to model and verify the behavior of a system before converting it into actual hardware. VHDL also supports concurrent system design, meaning multiple parts of a circuit can operate simultaneously. This makes it suitable for describing complex digital systems.

- **Introduction to Xilinx**

Xilinx is one of the leading companies in the field of programmable logic devices and is known for inventing the Field Programmable Gate Array (FPGA). The company provides a wide range of programmable hardware solutions used in industries such as telecommunications, automotive systems, consumer electronics, and embedded computing.

Xilinx offers several FPGA product families. Two of the most well-known families are the **Virtex series**, which is designed for high-performance applications, and the **Spartan series**, which targets cost-sensitive and high-volume applications. These FPGA families have evolved over multiple generations, offering improved performance, reduced power consumption, and increased logic capacity.

Xilinx also provides software tools that support the design, simulation, synthesis, and implementation of digital circuits. Earlier, the **ISE Design Suite** was widely used for FPGA design. This suite supported hardware description languages such as VHDL and Verilog, along with tools for synthesis, place-and-route, verification, and bitstream generation used to configure FPGA devices.

In this project, the **Vivado Design Suite (Version 2025.2)** was used to design and simulate the VHDL-based 8-bit Arithmetic Logic Unit (ALU). The tool was used to write the code, create a testbench, and verify the correct functionality of the design through simulation.

- **Design Methodology**

The objective of this project is to design and verify an **8-bit Arithmetic Logic Unit (ALU)** capable of performing sixteen arithmetic and logical operations using VHDL. The design was developed and simulated using the **Xilinx Vivado** environment. The ALU forms a fundamental component of modern processors and digital systems, as it performs essential computations required during program execution.

The design methodology followed in this project is based on a standard digital hardware design flow commonly used in VLSI and FPGA-based development. In this process, the system functionality is first analyzed and translated into a hardware model using a hardware description language. The model is then verified through simulation before moving to implementation.

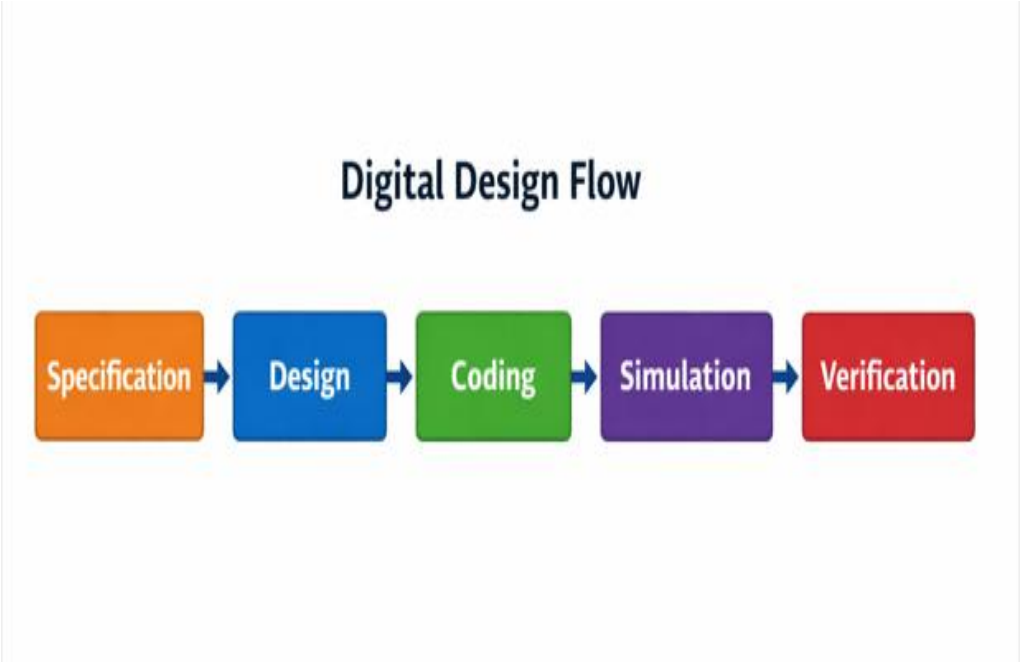
The first step in the design process was identifying the required operations and defining the system inputs and outputs. After the specifications were finalized, an architectural model of the ALU was created. The architecture defines how different arithmetic and logical units are integrated and controlled using selection signals.

The next stage involved writing the VHDL code describing the ALU behavior at the Register Transfer Level (RTL). RTL modeling focuses on the flow of data between registers and the logical operations performed on that data. The design was structured in a modular way to improve readability and maintainability.

After completing the design code, a testbench was developed to verify the correct functionality of the ALU. The testbench generates various input combinations and checks whether the output matches the expected results. Simulation was performed in Vivado to analyze the waveform outputs and confirm correct operation.

The overall methodology followed in this project can be summarized as:

1. Problem analysis and definition of ALU operations
2. Development of ALU architecture
3. RTL modeling using VHDL
4. Creation of testbench for verification
5. Behavioral simulation using Vivado
6. Validation of output waveforms and results



- **<u>8-Bit ALU Architecture</u>**

The Arithmetic Logic Unit is a combinational digital circuit that performs arithmetic and logical operations on binary data. It is one of the core building blocks of a processor and is responsible for performing calculations required during instruction execution.

In this project, an **8-bit ALU** was designed with two input operands and a control signal that determines which operation is executed. The ALU processes binary data and generates an output based on the selected operation.

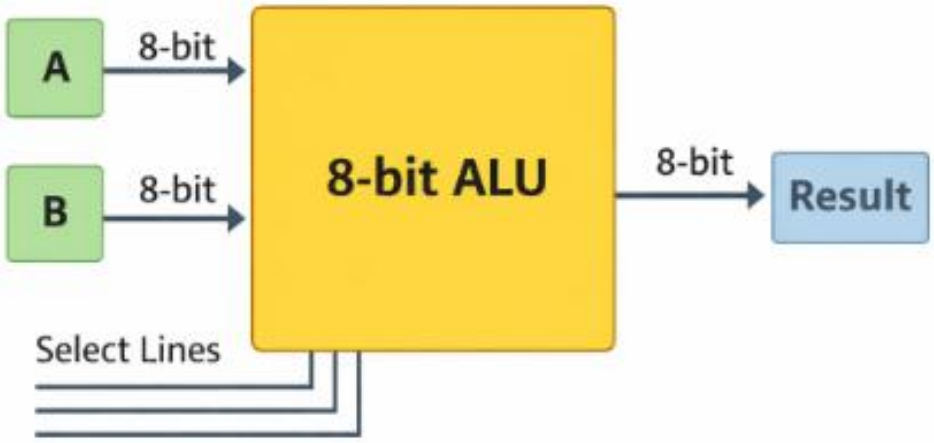
Inputs

- Operand A (8-bit input)
- Operand B (8-bit input)
- Select lines / Control signal (4-bit)

Outputs

- Result (8-bit output)
- Carry / Borrow flag (optional depending on operation)

The select lines act as control signals that determine which functional unit inside the ALU is activated. Based on the value of the control signal, the ALU performs the required arithmetic or logical operation.



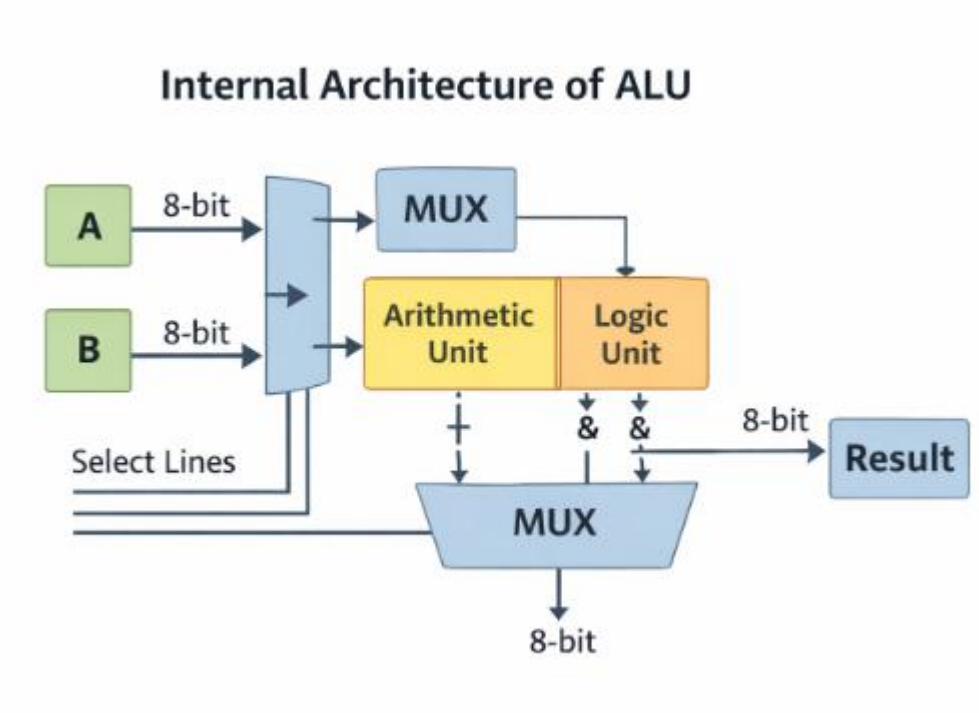
- **<u>Internal Working of the ALU</u>**

The ALU is internally composed of multiple functional units such as an adder, subtractor, logical operation unit, and shifter. These units operate in parallel, and a multiplexer selects the appropriate output depending on the control signal.

This design approach improves efficiency because all operations are available simultaneously, and the required output is selected instantly using control logic. Such architectures are commonly used in modern processor design.

The ALU architecture used in this project includes:

- Arithmetic Unit
- Logic Unit
- Shift Unit
- Operation Selector (Multiplexer)



- **<u>ALU Operations</u>**

The designed ALU supports **16 operations** controlled by a 4-bit selection input.

Select	**Operation**
0000	Addition
0001	Subtraction
0010	AND
0011	OR
0100	XOR
0101	NAND

Select	Operation
0110	NOR
0111	XNOR
1000	Shift Left
1001	Shift Right
1010	Increment
1011	Decrement
1100	NOT A
1101	Transfer A
1110	Transfer B
1111	Clear

These operations enable the ALU to perform the basic computations required in digital systems and microprocessors.

Arithmetic Operations

Arithmetic operations form the core functionality of the ALU. These operations include addition, subtraction, increment, and decrement.

Addition

Addition is performed by adding the two input operands bit by bit. The operation follows standard binary arithmetic rules

where the carry from one bit position propagates to the next higher bit position.

The adder circuit is responsible for generating the sum and carry outputs. In digital systems, addition is typically implemented using a chain of full adders.

Subtraction

Subtraction is implemented using the two's complement method. Instead of designing a completely separate subtractor circuit, subtraction can be achieved by adding the two's complement of the second operand to the first operand.

This approach simplifies the hardware design because the same adder circuit can be reused.

Mathematically,

A − B = A + (Two's complement of B)

This method is widely used in processors and digital systems.

Logical Operations

Logical operations manipulate bits individually and are widely used in digital decision-making processes.

AND Operation

The AND operation produces an output of 1 only when both input bits are 1.

OR Operation

The OR operation produces an output of 1 if any one of the input bits is 1.

XOR Operation

The XOR operation produces an output of 1 when the two input bits are different.

NOT Operation

The NOT operation inverts the input bits.

These operations are important for implementing control logic, comparisons, and masking operations in processors.

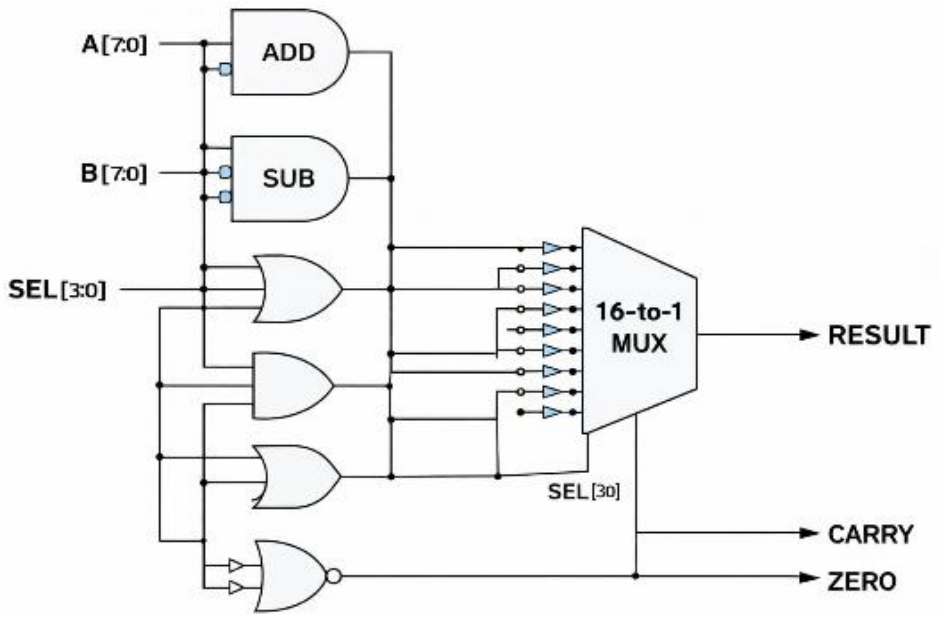
Shift Operations

Left Shift

In a left shift operation, all bits move one position toward the most significant bit. The least significant bit position is filled with zero.

Right Shift

In a right shift operation, bits move toward the least significant position while the most significant bit is filled with zero.



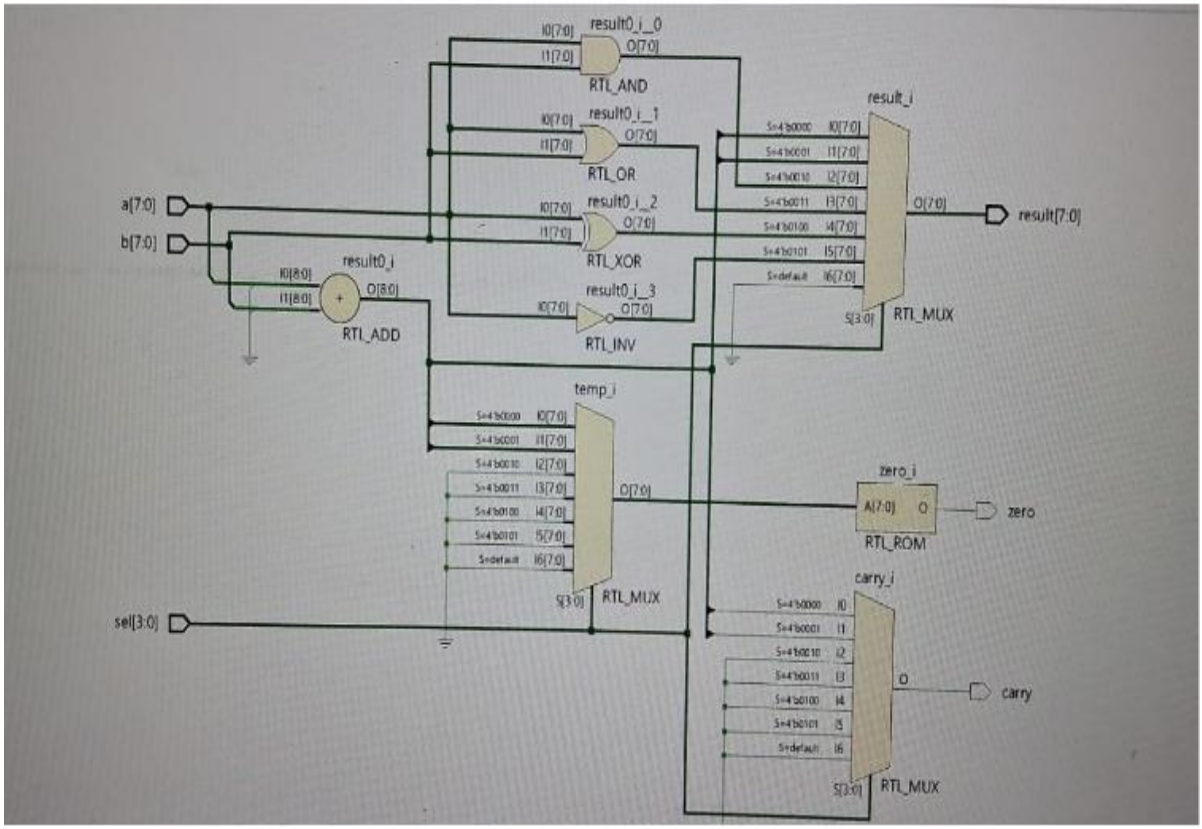
VHDL Implementation

The ALU was implemented using VHDL by defining the entity, architecture, inputs, outputs, and internal logic. The behavior of the ALU was described using conditional statements based on the select signal.

Each operation was implemented as part of a selection structure that determines which output is assigned to the result signal. This approach allows the ALU to perform different operations depending on the control input.

The modular nature of VHDL makes it easier to modify and extend the design in future implementations.

RTL design



Design code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity alu_8bit is
   Port (
      a      : in  STD_LOGIC_VECTOR(7 downto 0);
      b      : in  STD_LOGIC_VECTOR(7 downto 0);
      sel    : in  STD_LOGIC_VECTOR(3 downto 0);
      result : out STD_LOGIC_VECTOR(7 downto 0);
      carry  : out STD_LOGIC;
      zero   : out STD_LOGIC
   );
end alu_8bit;

architecture Behavioral of alu_8bit is
begin

process (a,b,sel)
   variable temp : unsigned(8 downto 0);
begin
```

```
case SEL is

    when "0000" =>  -- ADD
        temp := unsigned(a) + unsigned(b);
        result <= std_logic_vector(temp(7 downto 0));
        carry  <= temp(8);

    when "0001" =>  -- SUB
        temp := unsigned(a) - unsigned(b);
        result <= std_logic_vector(temp(7 downto 0));
        carry  <= temp(8);

    when "0010" =>  -- AND
        temp := (others => '0');
        result <= a and b;
        carry  <= '0';

    when "0011" =>  -- OR
        temp := (others => '0');
        result <= a or b;
        carry  <= '0';

    when "0100" =>  -- XOR
        temp := (others => '0');
        result <= a xor b;
```

```
            carry  <= '0';

        when "0101" =>  -- NOT A
            temp := (others => '0');
            result <= not a;
            carry  <= '0';

        when others =>
            temp := (others => '0');
            result <= (others => '0');
            carry  <= '0';

    end case;

    -- ZERO flag calculation (correct way)
    if temp(7 downto 0) = 0 then
        zero <= '1';
    else
        zero <= '0';
    end if;

  end process;
end Behavioral;
```

Testbench Design

A testbench was developed to verify the correct operation of the ALU. The testbench applies various combinations of input values and control signals to the ALU and observes the output.

The purpose of the testbench is to simulate real working conditions and ensure that the ALU performs correctly for all operations.

Simulation tools generate waveform outputs that show how signals change over time. These waveforms help in debugging and validating the design.

Testbench Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity alu_8bit_tb is
end alu_8bit_tb;

architecture Behavioral of alu_8bit_tb is
```

```
-- Component Declaration
component alu_8bit
   Port (
      a      : in  STD_LOGIC_VECTOR(7 downto 0);
      b      : in  STD_LOGIC_VECTOR(7 downto 0);
      sel    : in  STD_LOGIC_VECTOR(3 downto 0);
      result : out STD_LOGIC_VECTOR(7 downto 0);
      carry  : out STD_LOGIC;
      zero   : out STD_LOGIC
   );
end component;

-- Signals
signal a      : STD_LOGIC_VECTOR(7 downto 0);
signal b      : STD_LOGIC_VECTOR(7 downto 0);
signal sel    : STD_LOGIC_VECTOR(3 downto 0);
signal result : STD_LOGIC_VECTOR(7 downto 0);
signal carry  : STD_LOGIC;`
signal zero   : STD_LOGIC;

begin

-- Instantiate ALU
```

```
UUT: alu_8bit
    port map (
        a => a,
        b => b,
        sel => sel,
        result => result,
        carry => carry,
        zero => zero
    );

-- Stimulus Process
process
begin

    -- ADD
    a <= "00001010"; -- 10
    b <= "00000101"; -- 5
    sel <= "0000";
    wait for 10 ns;

    -- SUB
    sel <= "0001";
    wait for 10 ns;
```

```
    -- AND
    sel <= "0010";
    wait for 10 ns;

    -- OR
    sel <= "0011";
    wait for 10 ns;

    -- XOR
    sel <= "0100";
    wait for 10 ns;

    -- NOT
    sel <= "0101";
    wait for 10 ns;

    wait;
end process;

end Behavioral;
```

Simulation and Verification

Simulation and verification are essential stages in digital system design. Before implementing a design on hardware, it is important to confirm that the logic performs the intended operations correctly. In this project, simulation was carried out using the **Xilinx Vivado** simulation environment.

After completing the VHDL code for the 8-bit Arithmetic Logic Unit, a testbench was created to apply different input combinations and verify the correctness of the output. The testbench generates input signals for operands A and B along with the select lines that determine the operation performed by the ALU.

The simulation process involved the following steps:

1. Writing the ALU design in VHDL.
2. Creating a testbench file to provide input stimuli.
3. Running behavioral simulation in Vivado.
4. Observing waveform outputs in the simulation window.
5. Verifying that the output matches the expected results.

During simulation, multiple test cases were applied to ensure that each of the sixteen operations works correctly. The waveform viewer in Vivado displays the transitions of input and output signals with respect to time. By analyzing these waveforms, it is possible to confirm the correctness of the design.

Simulation allows designers to identify errors in logic before hardware implementation. This reduces development time and ensures that the final design behaves as expected.

Results and Output Waveforms

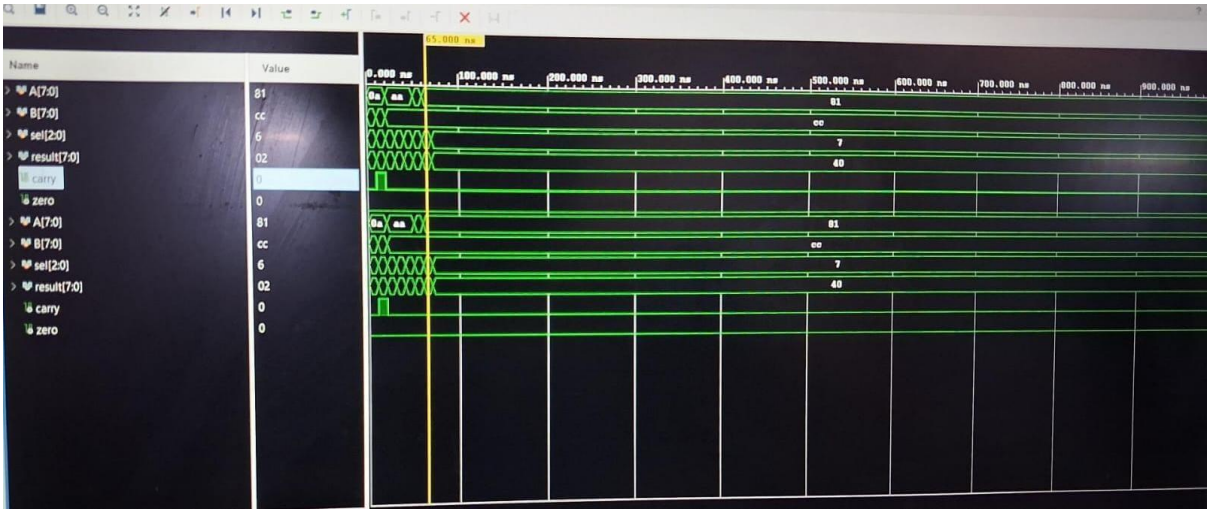
The simulation results demonstrate that the designed 8-bit ALU successfully performs all the specified arithmetic and logical operations. The output generated by the system corresponds correctly to the given input operands and control signals.

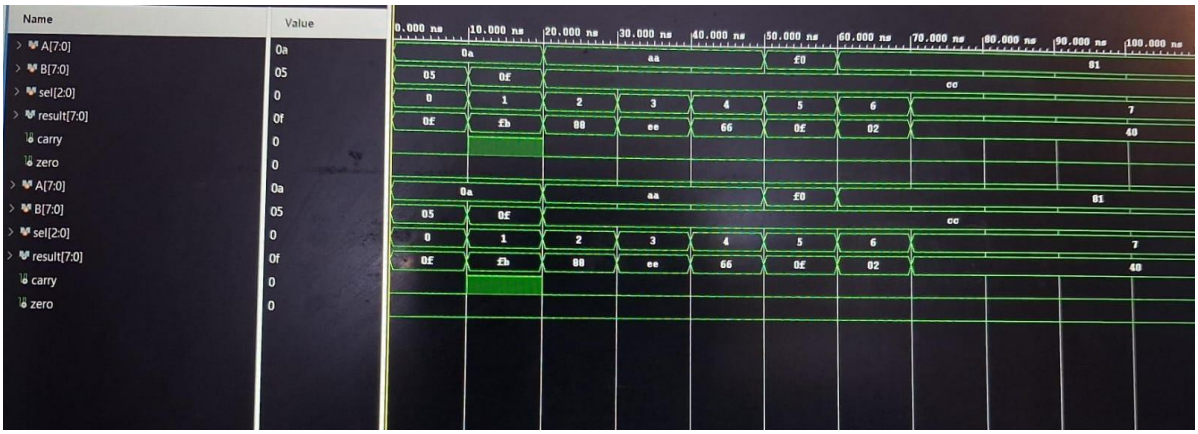
For each value of the select line, the ALU performs the intended function such as addition, subtraction, logical operations, or shifting. The waveform output confirms that the ALU transitions correctly between operations and produces accurate results.

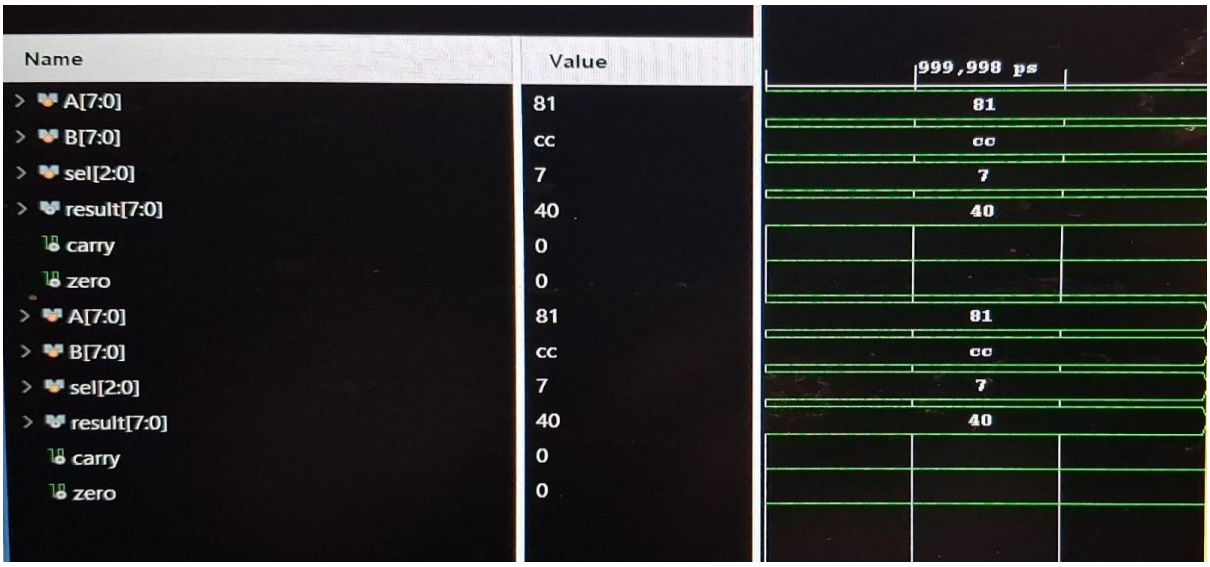
The following observations were made during simulation:

• The addition operation correctly generates the sum of the two input operands.
• The subtraction operation produces the correct difference using binary arithmetic.
• Logical operations such as AND, OR, XOR, and NOT behave as expected.

The waveform analysis confirms that the ALU design is functionally correct and stable. The output signals change according to the control inputs without any unexpected behaviour.


Name
Value
A[7:0]
B[7:0]
sel[2:0]
result[7:0]
carry
zero
A[7:0]
B[7:0]
sel[2:0]
result[7:0]
carry
zero
81
cc
6
02
0
0
81
cc
6
02
0
0


Name
Value
A[7:0]
B[7:0]
sel[2:0]
result[7:0]
carry
zero
A[7:0]
B[7:0]
sel[2:0]
result[7:0]
carry
zero
0a
05
0
0f
0
0
0a
05
0
0f
0
0


Name
Value
A[7:0]
B[7:0]
sel[2:0]
result[7:0]
carry
zero
A[7:0]
B[7:0]
sel[2:0]
result[7:0]
carry
zero
81
cc
7
40
0
0
81
cc
7
40
0
0
999,998 ps
81
cc
7
40
81
cc
7
40

FPGA

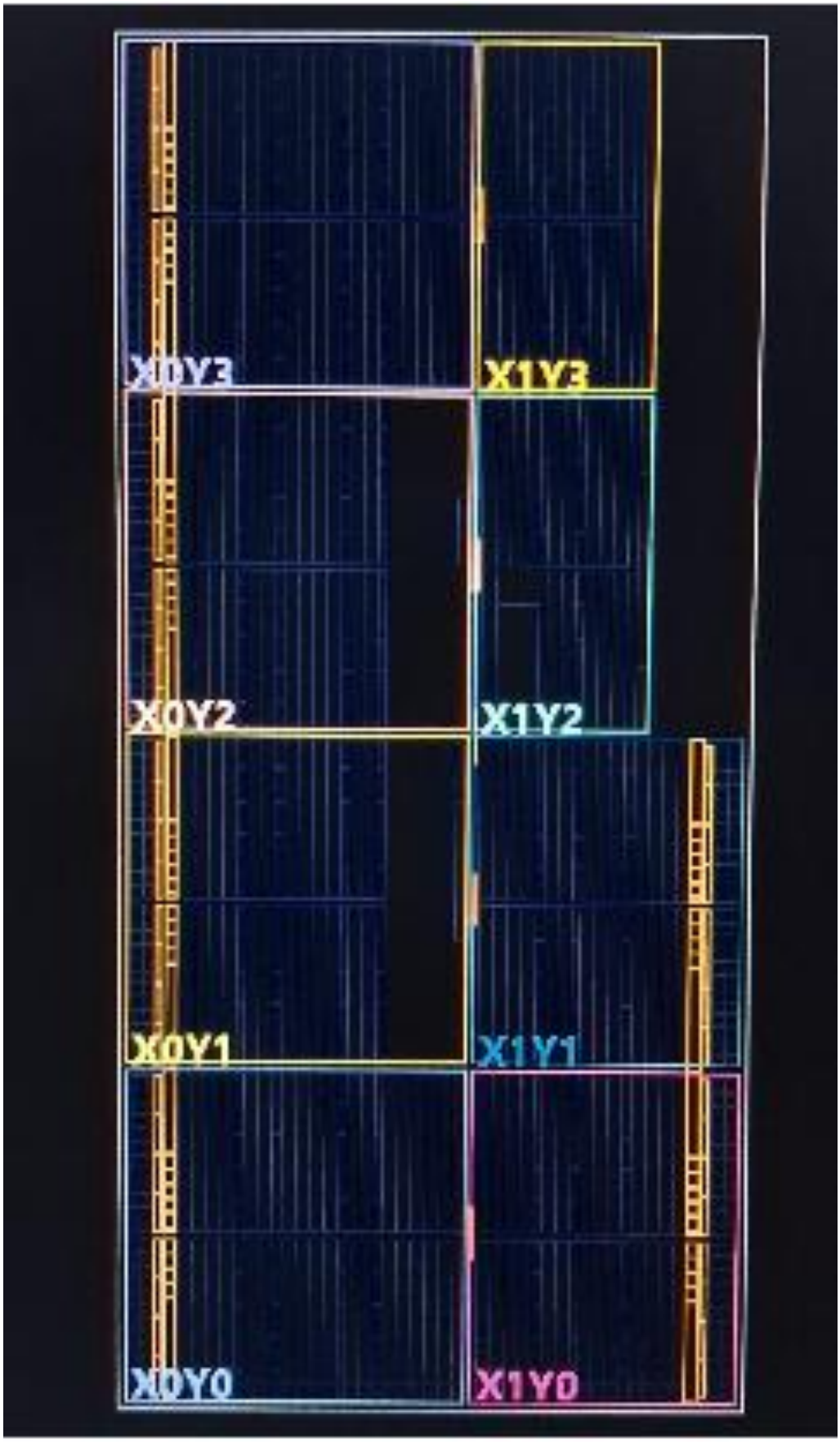
Although the project focuses on the design and simulation of the ALU, the developed system can be further extended and implemented on actual FPGA hardware. Since the design is written in VHDL and verified through simulation, it can be synthesized and programmed onto an FPGA device.

Future improvements that can be made include:

- Implementing the ALU on an FPGA development board.
- Increasing the ALU width from 8-bit to 16-bit or 32-bit.
- Adding more advanced arithmetic operations such as multiplication and division.
- Integrating the ALU with other processor components such as registers and control units.
- Optimizing the design for speed, area, or power consumption.
- Implementing pipelining techniques to improve performance.

Such enhancements would allow the design to be used as part of a complete processor architecture. FPGA implementation would also allow real-time testing and performance analysis.

BELOW IS THE SYNTHESISIZED FPGA BASED CHIP DESIGN:



- **<u>Conclusion</u>**

In this project, an **8-bit Arithmetic Logic Unit (ALU)** capable of performing sixteen operations was successfully designed and verified using VHDL. The design was developed and simulated using the **Xilinx Vivado** environment.

The project helped in understanding the working of digital arithmetic circuits and the use of hardware description languages in modern system design. Through simulation and verification, it was confirmed that the ALU performs all operations correctly for various input combinations.

Debtanu Mondal

(PROJECT COORDINATOR)